

A Model-Based Approach to Shooting on the Move - Team 5987, Galaxia

This season our team decided that on our turret'd robot Shoot on the move is a must, after discussion of how to implement it a couple of days after kickoff we decided we want to go with the model based approach and build a functional physics simulation which will be the base of our shoot on the move.

Before diving into the details and explanations, it's worth noting that the entire project is open source and available on GitHub. It is also adaptable to any robot that shoots balls with minimal or no spin, making it broadly applicable.

[Simulation & Analysis Repository](#)
[Our Robot Code](#)

Defining the problem

To start off we needed to define the problem. First we'll make some common language so it'll be easier to understand.

Coordinate system:

The x -axis points inward, toward the hub

The y -axis is perpendicular to the x -axis

Variables:

d - **turret** distance from the center of the hub

v_x - **turret** velocity along the X-axis

v_y - **turret** velocity in the axis Y-axis

Outputs:

Each of our outputs should take the given d, v_x, v_y and give us back a result.

θ - desired hood angle

$\Delta\phi$ - turret rotation (delta)

v_{ball} - ball exit velocity

We define three functions that map the turret state to the desired outputs.

Given the turret distance (d) from the hub and its velocity components (v_x) and (v_y), the hood angle is computed as

$$\theta = f_{\theta}(d, v_x, v_y),$$

the turret rotation (delta) is

$$\Delta\phi = f_\phi(d, v_x, v_y),$$

and the ball exit velocity is

$$v_{\text{ball}} = f_v(d, v_x, v_y).$$

To determine the three functions, we implemented a simulation capable of solving constrained problems. Essentially, we iterate over all possible combinations of d , v_x , and v_y within predefined ranges. For each combination, the simulation computes the optimal solution according to the constraints (to be defined later). The results are then saved in a comprehensive CSV file, which will subsequently be analyzed to derive the three required functions. The methodology for this analysis will also be explained in detail later.

Implementing the Simulation

To start with we decided it'd be easier to have some sort of base for our simulation. After some searching we found [Sleipnir](#), which is a reverse-mode autodiff library, interior-point method, and NLP solver DSL.

Looking through their FRC 2022 example gave us a pretty solid base to start off.

Since we are essentially generating a huge lookup table we decided to write the entire simulation part of the project in c++. Speed and compiler optimizations being the main factor for this.

We then adapted their example to improve readability and tailor it to our specific use case. Several configurations were added to the project, including support for turret offsets, hood angle constraints, and the ability to define the iteration ranges. The main iteration loop was implemented, along with other supporting features. Finally, a small GUI was added to visualize the simulation results.

Fuel Constants

In order to take into account the [Drag Equation](#), we needed three real world parameters of the ball - the ball radius, the ball mass, and its drag coefficient.

While we had no issues getting the ball's radius and mass from the specifications, determining a precise drag coefficient proved slightly more complex.

Finding the friction coefficient

In order to find C_d , we used the drag equation and Newton's seconds law to make the following equation:

According to Newton's seconds law:

$$F = ma$$

As $\frac{dv}{dt}$ is α , and $\sum F$ is mg minus the drag equation:

$$m \frac{dv}{dt} = mg - \frac{1}{2} \rho A C_D v^2$$

After integrating, we get the following equations:

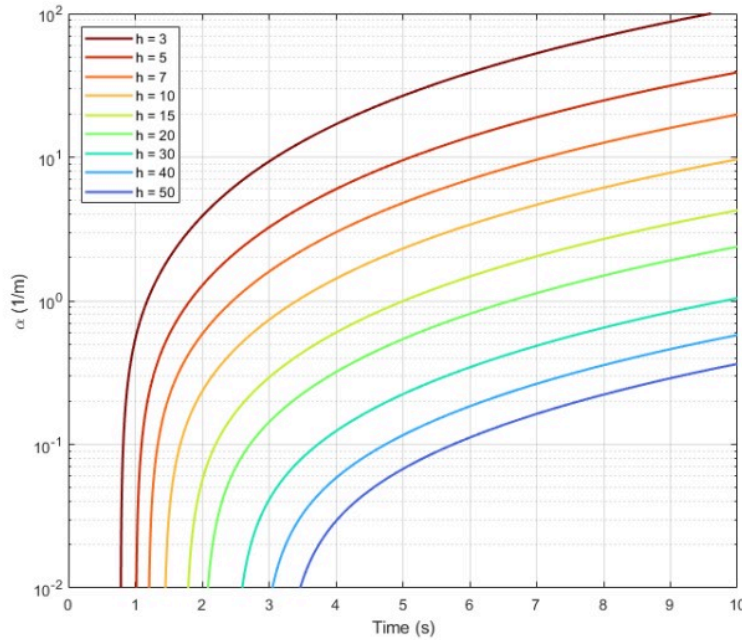
$$t = \frac{1}{\sqrt{g\alpha}} \cosh^{-1}(\exp(\alpha \cdot h))$$

$$\alpha = \frac{\pi}{2m} \rho r^2 C_D$$

Which we could plug t , h , and m , in order to find C_d .

So we conducted an experiment - We dropped the FUEL, from 5 meters high, and filmed it to find the time it took until it hit the ground.

Because the equation doesn't have an algebraic solution, we need to plot it, and find the intersection, to find our C_d .



We can now find C_D using the definition of α :

$$C_D = \frac{2\alpha \cdot m}{\pi \rho \cdot r^2} \quad (23)$$

At the end, we found the drag coefficient of the **FUEL** game element to be **0.63**.

The constraints

As solving this problem requires a cost-function approach, we first define the constraints on the variable matrix, and then we use Sleipnir to find the optimal solution according to the constraints.

Basic Physical Constraints

We need to supply the optimizer with basic physical constraints to prevent it from exploring mathematically valid but physically impossible solutions:

- Time of Flight: Time must flow forward, preventing some impossible solutions from occurring. $t_{of} \geq 0$
- Initial Position: The trajectory must begin at the robot's position. $\mathbf{p}_0 = \mathbf{p}_{shooter}$
- Final Position: The end of the trajectory must hit the **HUB**'s center. $\mathbf{p}_{N-1} = \mathbf{p}_{target}$
- Free Fall: The **FUEL** must be falling downward along the Z-axis, when it enters the **HUB**.
 $v_{z,N-1} < 0$
- Physical Hood Limits: The launch angle θ , calculated from the muzzle velocity vector, must stay within the hood mechanical constraints(15° to 35°).

$$\theta = \arctan\left(\frac{v_{z,0}}{\sqrt{v_{x,0}^2 + v_{y,0}^2}}\right) \quad \theta_{min} \leq \theta \leq \theta_{max}$$

Drag Equation and Runge-Kutta 4

Because the drag equation outputs the state derivatives, velocity and acceleration, at any given time, we must integrate them to determine the actual drag influenced trajectory.

As the acceleration of the fuel is based on the current velocity and direction, the system forms a non-linear ordinary differential equation (ODE). Because of that, we used the Runge-Kutta 4 algorithm to integrate the drag equation.

$$\frac{d\mathbf{p}}{dt} = \mathbf{v}$$

$$\frac{d\mathbf{v}}{dt} = -\mathbf{g} - \left(\frac{\frac{1}{2}\rho C_D A |\mathbf{v}|^2}{m}\right) \frac{\mathbf{v}}{|\mathbf{v}|}$$

We get a pair of ODEs. After integrating via Runge-Kutta 4, we get the drag and gravity compensated states.

Choosing The Objective Function

When we first implemented the simulation, our initial objective function aimed to minimize the time of flight (ToF) of the ball. However, when visualizing the results in the GUI, we observed that this approach consistently produced very low, flat trajectories with attack angles close to zero, essentially the opposite of the behavior we wanted.

After experimenting with different formulations, we switched our objective function to minimize the shooter's initial launch velocity relative to the robot. This adjustment produced high-arching trajectories, which are preferable because they provide greater margin for error and make it easier to hit the target reliably.

Attack Angle Constraints

We realized that the attack angle(the angle at which the ball enters the HUB) is a critical factor in our shot success. We introduced the following constraint:

$$\arctan \left(-v_z, \sqrt{v_x^2 + v_y^2} \right) > \alpha_{\min \text{ attack}}$$

However, we quickly discovered that while this static constraint produced accurate shots from mid-range, it failed to perform well at very close or very long distances. To fix this, we developed a function to dynamically calculate the minimum attack angle based on the robot's distance from the **HUB**.

Initially, we experimented with exponential equations of the following form (where d represents distance):

$$\arctan \left(-v_z, \sqrt{v_x^2 + v_y^2} \right) > f(d)$$

$$f(d) = a \cdot e^{b(d-c)} + n$$

While this approach worked fairly well, the parameters were incredibly difficult to tune, making the process tedious and impractical.

Needing a more intuitive tuning method, we attempted to use basic if statements to construct a synthesized step function. Unfortunately, this approach yielded a non-continuous output data, which made it impossible to mathematically analyze and approximate with a polynomial function (as explained later in this document).

Ultimately, we settled on using a sum of sigmoid functions to generate a smooth, continuous step function:

$$f(d) = \sum_{i=1}^n \frac{h_i}{1 + e^{-k_i(d-d_i)}} + C$$

(Where h_i represents the height of each step, k_i determines the steepness of the transition, d_i is the distance threshold for the step, and C is a baseline constant.)

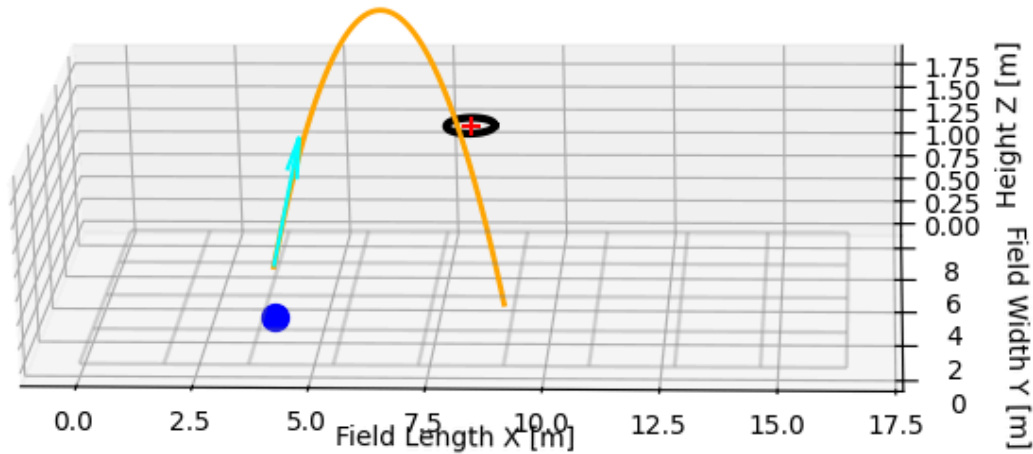
We wrote a simple script to generate this function automatically, providing us with an easy way to manually tune the ideal attack angle for any given distance.

Implementing this continuous step function resulted in a highly optimized pitch angle for every shot, significantly increasing our overall shot success rate.

Finding the optimal hood angle ranges via simulation

Once we had the basic simulation up and running, we ran it without constraining the hood angles to explore the full range of possible trajectories. By analyzing the resulting data, we identified the minimum and maximum viable angles, which showed that a range of 15° to 35° is optimal for

shooting from the corner. This range represents the most effective hood angles for successfully delivering fuel into the hub.



Too slow

Once the basic simulation was up and running, we realized that, despite being relatively optimized and implemented in C++, generating the output CSV still took several long hours. Since we were making frequent changes during the early stages of the season, this became a significant time bottleneck.

Multithreading

To address the long computation times, we implemented multithreading, enabling the program to utilize all available CPU threads. Each thread was assigned a sub-range of the input space, and once all threads completed their calculations, the results were merged and written into a single CSV file. This optimization reduced the total computation time from around five hours to approximately one hour, greatly improving our workflow during the early stages of development.

Compiler Optimizations

After enabling the [-O3](#) compiler optimization flag, the runtime of our simulation was reduced dramatically, dropping from around one hour to just one minute. This optimization allowed the program to run much faster without any changes to the underlying code, making it much easier and practical for iterative testing and development.

Analyzing the data

Once the simulation results were collected, we analyzed the data to derive usable functions. This involved fitting polynomial functions to match the lookup table (LUT) data and determining the correlation between linear velocity and angular velocity. These analyses allowed us to convert the raw simulation outputs into practical formulas for controlling the turret, the hood and the flywheels.

For that, we created a [Python Jupyter Notebook](#) that takes the data from the C++ simulation.

We used a few libraries to handle the data:

- [scikit-learn](#) - used for generating the linear and polynomial functions from the data.
- [pandas](#) - used to load and handle the simulation data.
- [plotly](#) and [seaborn](#) - used for visualizing the data and plotting.

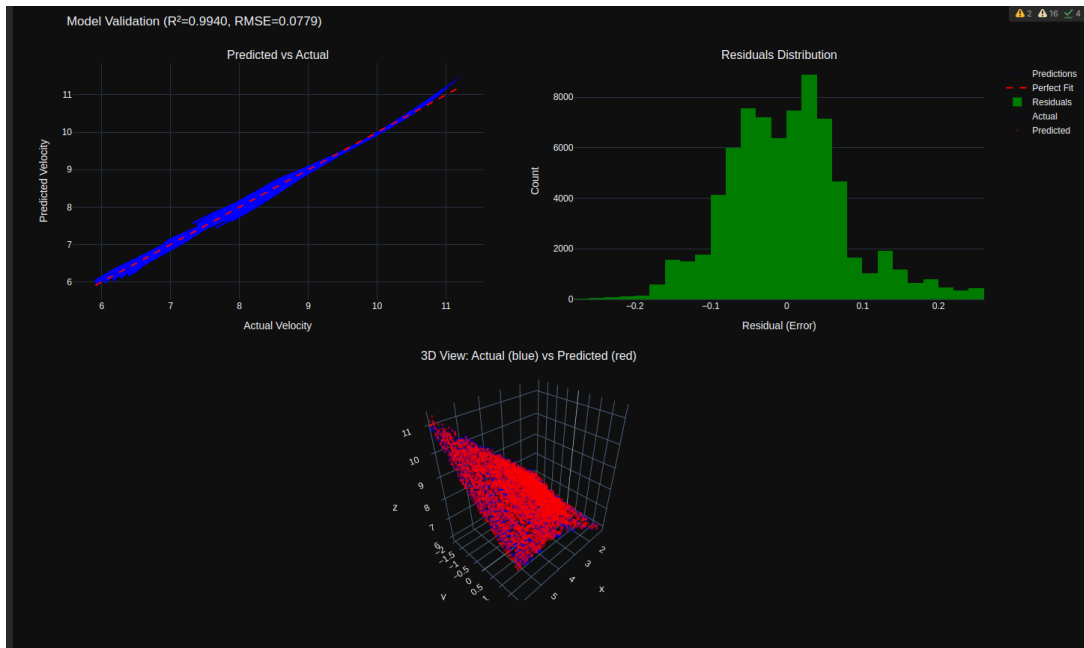
Our pipeline is as follows:

The process begins by loading the raw simulation data using the pandas library as a CSV file. Before performing any calculations or analyzing, we use seaborn and plotly to visualize the data, ensuring all data is valid.

We then use scikit-learn to apply the mathematical models, fitting polynomial curves to our LUT. This step creates highly accurate, continuous functions that control the turret, hood, and flywheel.

To make these formulas practical for the robot's codebase, we wrote custom helper functions that automatically translate the resulting mathematical expressions into formatted Java methods. Finally, the pipeline generates all of these methods and exports them directly into a single file called the ShotCalculator.

This automated pipeline ensures our robot runs on the most up-to-date physics model straight from the simulation.

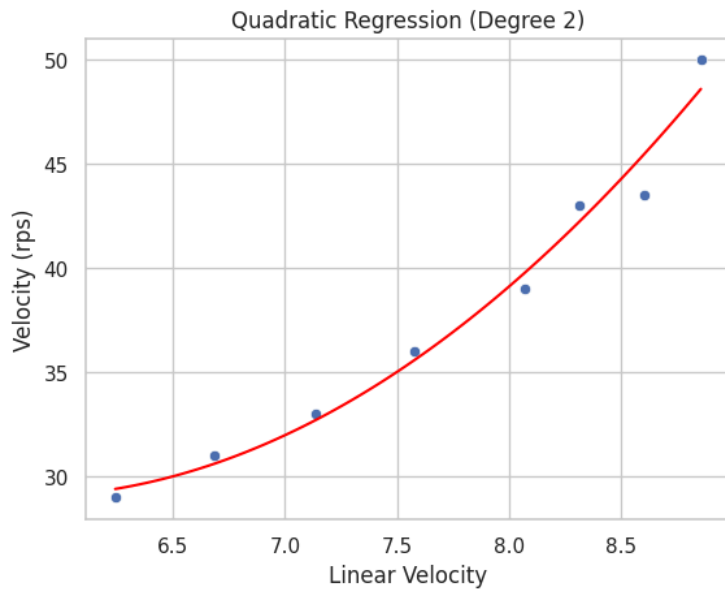


Finding the linear velocity to angular velocity correlation

Map Matching Method

Because our simulation outputs linear velocity, we faced a challenge in controlling our motors, which require angular velocity commands. Since real-world conditions introduce friction and prevent the flywheels from achieving pure rolling on the balls, the basic formula $\omega = \frac{v}{r}$ isn't accurate enough.

Calculating all of these physical variables mathematically is practically impossible, so we decided to find this relationship through calibration. First, we mapped distances directly to successful angular velocities by taking static shots using the simulation's hood angles. Then, we matched these physical angular velocities to the simulation's target linear velocities for those same distances. This gave us a clear correlation between the theoretical linear velocity and the practical angular velocity, which turned out to be a 2nd-degree polynomial function.



Depth Camera Analysis Method

To determine the correlation between linear and angular velocity, we also evaluated a vision-based approach using the Intel RealSense D435 Depth Camera. This camera utilizes an onboard infrared projector to capture accurate RGB-D (color and depth) data. We recorded the ball's trajectory at the optimal hood angle and processed the resulting ROS bags using a custom pipeline built with the librealsense SDK.

By applying basic image processing, we tracked the yellow ball in the RGB feed and mapped its 2D position to 3D spatial coordinates for each frame. With the 3D trajectory established, we used the SymPy library to calculate the time derivative of the position data across each axis, yielding the ball's linear velocity. Repeating this process across multiple trials while changing the angular velocity allowed us to map the calculated linear velocities to their corresponding angular velocities. Ultimately, however, this method was abandoned. The resulting correlation lacked accuracy, and the process was too time-consuming to be practical.

Robot Code Usage & Optimizations

After implementing our three functions and the linear-to-angular velocity conversion, the next step was to integrate them into our robot code.

Since our simulation uses a turret-oriented approach, we now need to define a **turret-oriented speed variable**, consisting of v_x, v_y which will then be passed into our functions.

Calculating Turret Speeds

Initially, we attempted to calculate the turret-oriented speeds by simply taking the robot-oriented speeds and rotating them according to the turret's position, aligning with our axis convention. This approach worked reasonably well, but it revealed a significant problem: when the driver was turning the robot, our robot frequently missed targets.

Why did this happen? The issue is that we weren't accounting for the robot's rotational motion. When the robot spins, its angular velocity ω contributes additional velocity to the turret, affecting v_x and v_y depending on the turret's position relative to the center of rotation. Ignoring this effect caused discrepancies between the expected and actual turret motion during turns.

Accounting for Turret Tangential Velocity

To fix the issue, we incorporated the **tangential velocity of the turret** caused by the robot's rotation. Physically, any point on a rotating body moves in a circle around the center of rotation. The linear (tangential) velocity of that point is given by

$$v_{\text{tangential}} = \omega * r$$

where ω is the angular velocity of the robot, and r is the distance from the robot's center to the turret. The direction of this tangential velocity is always **perpendicular to the radius vector** from the center of rotation to the turret.

By calculating this tangential component and adding it to the turret-oriented velocities, we ensure that the turret's motion reflects both the robot's translation and rotation. This corrects the errors observed earlier because the turret now properly accounts for the additional velocity added by the robot spinning.

Here's the code for the above:

```
const val TURRET_RADIUS_TO_CENTER_ROBOT_METERS = 0.23868132
val TURRET_VELOCITY_ANGLE = 60.4426997.deg

fun getTurretTangentialVelocityFieldRelative(
    omegaRadiansPerSecond: Double
): Translation2d {
    val magnitude = omegaRadiansPerSecond * TURRET_RADIUS_TO_CENTER_ROBOT_METERS
    return Translation2d(
        -Math.cos(TURRET_VELOCITY_ANGLE[rad]) * magnitude,
        -Math.sin(TURRET_VELOCITY_ANGLE[rad]) * magnitude,
    )
}
```

Custom Turret Feedforward

Another challenge we encountered was that, even though the turret was receiving the “correct” setpoint, rapid changes in that setpoint caused the control to lag or overshoot. To address this, we implemented a **feedforward term** based on the rate of change of the setpoint.

Specifically, we calculate the **setpoint change rate** as

$$\frac{\Delta_{\text{setpoint}}}{\Delta t}$$

and multiply it by a **calibrated constant**. This feedforward contribution is then applied using the built-in `.withFeedForward` method on our position control request.

By anticipating the turret's motion, this approach improves responsiveness and reduces turret lagging in high robot velocities

Improving Accuracy

Even after implementing various improvements, we were still seeing a noticeable number of missed shots when our robot attempted to shoot while moving. One of the most effective changes we made was actually quite simple: constraining the driver.

In our current code, when the robot is shooting on the move towards the hub, the robot's maximum velocity is reduced to 2.0 meters per second. This limit prevents sudden accelerations and extremely high speeds, allowing the robot to maintain better control and significantly improves accuracy during dynamic shooting.

Implementation Links:

[**Simulator**](#) - Repository containing both the simulation code and related analysis tools used to model and test system behavior, written in C++.

[**Analysis Notebook**](#) - Jupyter Notebook used for data analysis and visualization, written in Python.

[**Robot Code**](#) - Main codebase that runs on the robot, written mostly in Kotlin.

[**SOTM Calculations Code**](#) - Contains all calculations required for shooting while the robot is in motion, including kinematics, prediction, and compensation across subsystems. Written in Kotlin.

[**Turret Calculations Code**](#) - Handles general turret-related computations, written in Kotlin.

[**Shoot On Move Mathematical Functions**](#) - Contains the mathematical functions for the hood, turret, flywheel and the Angular velocity to the linear velocity conversion, written in Java.

[**Realsense-based ballistic analyzer**](#) - Contains the Python code for generating the angular-to-linear correlation using the Intel Realsense depth camera.

[**Sleipnir**](#) - Reverse mode autodiff library, interior-point method, and NLP solver DSL for C++23 and Python

If you have any questions feel free to reach out and we'll happily help.